

AWS SERVERLESS MONITORING APPLICATION

Implementation Screenshot Documentation

Amazon S3 • Amazon CloudFront • Origin Access Control • AWS Lambda • DynamoDB

Purpose

This document organizes the screenshots from the original six-page project record into a professional implementation guide. Each screenshot is followed by detailed explanatory text describing what was configured, why the step matters, and how it fits into the overall serverless architecture.

Project outcome

The demonstrated workflow uses Amazon S3 for static frontend storage, Amazon CloudFront for content delivery, CloudFront Origin Access Control (OAC) for protected access to the S3 origin, and AWS Lambda for serverless backend/API logic. The project workflow also uses DynamoDB as the serverless data layer.

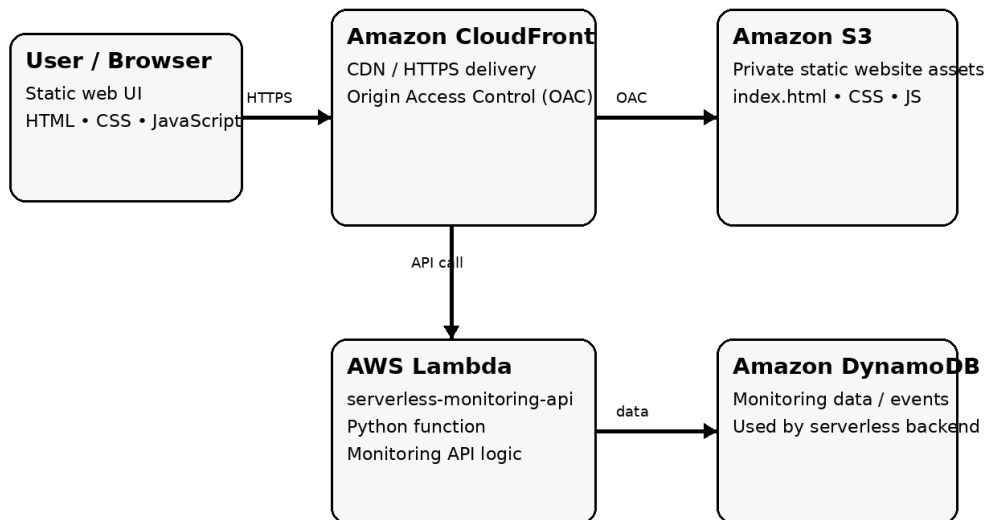
Prepared for GitHub / LinkedIn portfolio documentation

1. Architecture Overview

The architecture separates the presentation layer from the backend processing layer. The browser receives the static frontend through CloudFront. CloudFront uses OAC to access the S3 origin. The frontend can invoke serverless backend functionality implemented with Lambda, and the backend uses DynamoDB for application data.

Logical flow: User Browser → CloudFront → S3 (static assets) → Frontend API request → Lambda → DynamoDB.

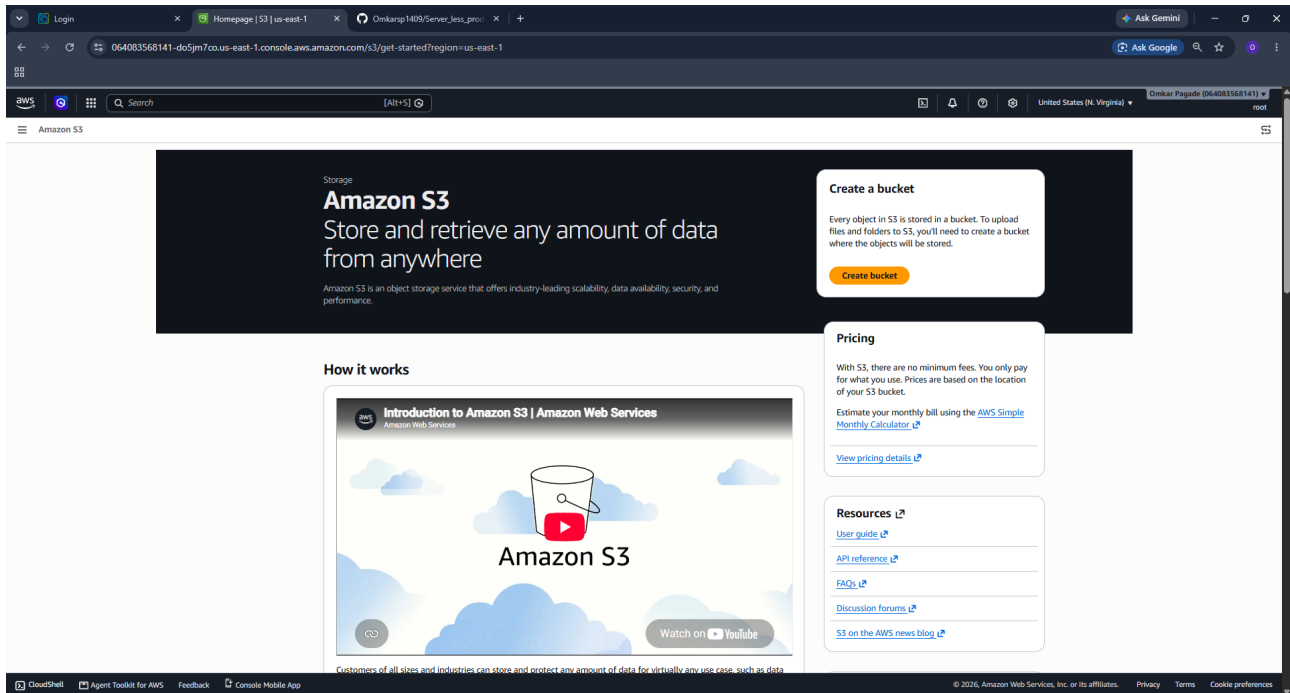
AWS Serverless Monitoring Architecture



Deployment demonstrated with Amazon S3, CloudFront, AWS Lambda and DynamoDB.

Figure 1 — High-level AWS serverless architecture used for the project.

2. Amazon S3 — Understanding the Service



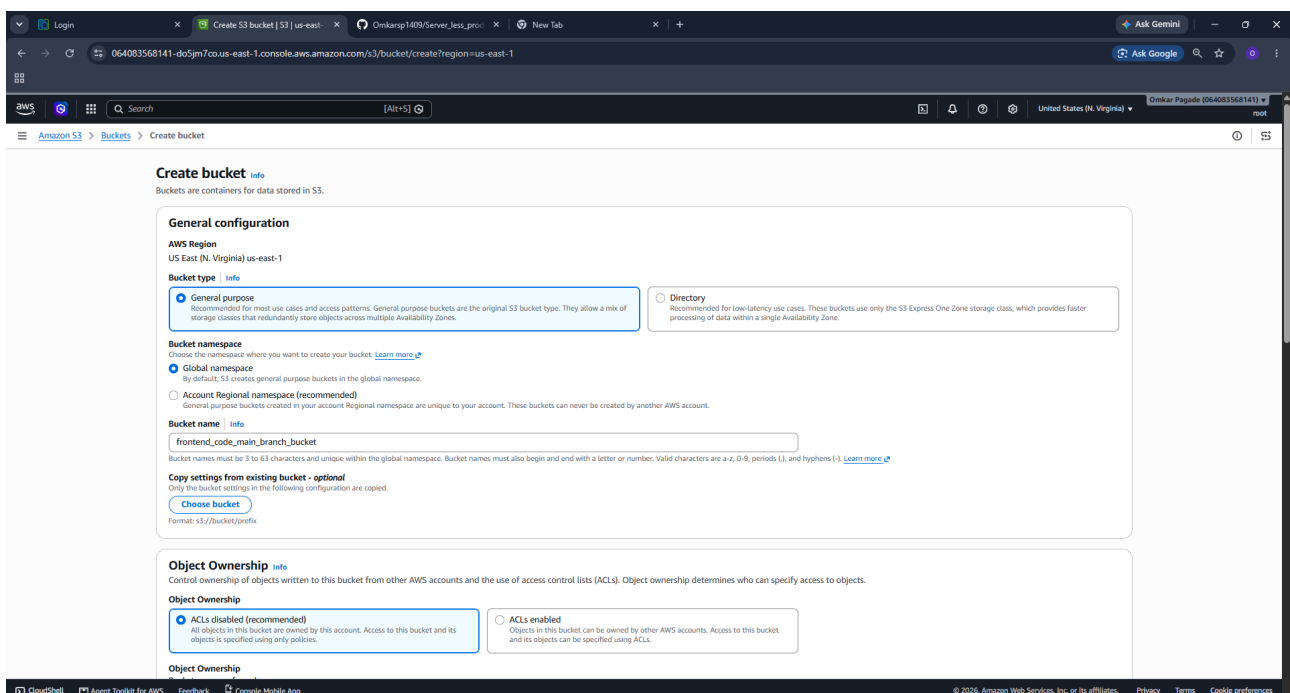
What this step does: Amazon S3 is used as the storage layer for the static frontend files. In this project, the bucket holds the website assets such as HTML, CSS and JavaScript.

Why it matters: Using S3 removes the need to maintain a traditional web server for static content. It is a managed storage service and works well as an origin for CloudFront.

What the screenshot shows: The screenshot shows the Amazon S3 service page and the beginning of the bucket creation workflow.

Expected result: The required S3 bucket becomes the storage location for the application's frontend assets.

3. S3 Bucket Creation — General Configuration



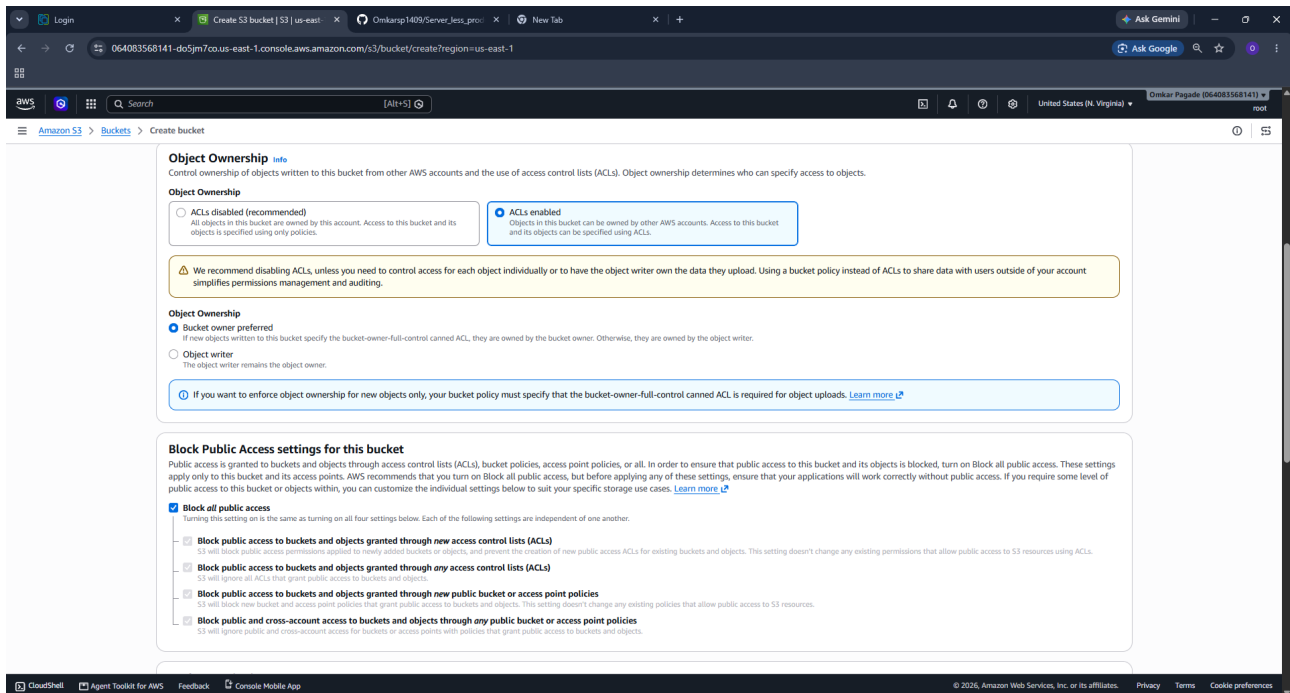
What this step does: The bucket creation screen is used to define the bucket name and basic configuration before the bucket is created.

Why it matters: The bucket is the logical container for the frontend objects. Naming and access configuration are important because the same bucket will later be referenced by CloudFront.

What the screenshot shows: The screenshot shows the S3 Create bucket interface with general configuration and object ownership/access-related settings.

Expected result: A dedicated bucket is created for the static frontend content.

4. S3 Object Ownership and Public Access Settings



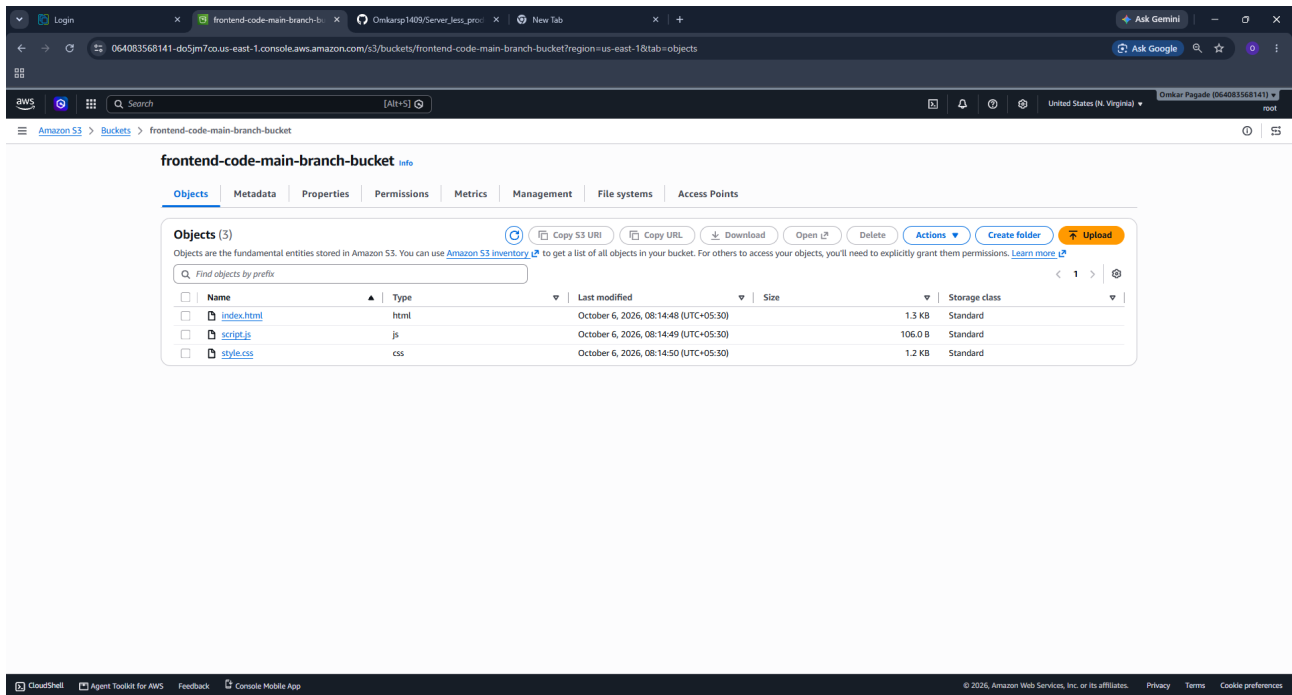
What this step does: The next S3 configuration screen contains object ownership and Block Public Access controls.

Why it matters: For a CloudFront + OAC architecture, the design goal is to keep the S3 origin protected and allow CloudFront to access it. This is preferable to exposing the bucket directly to the public internet.

What the screenshot shows: The screenshot shows Object Ownership and Block Public Access settings in the bucket creation workflow.

Expected result: The bucket is prepared for controlled access and later integration with CloudFront.

5. S3 Bucket — Frontend Objects Uploaded



What this step does: After bucket creation, the frontend files are uploaded into the S3 bucket.

Why it matters: The static website is composed of files rather than a running server process. S3 stores these objects reliably and makes them available to CloudFront as the origin.

What the screenshot shows: The screenshot shows the created bucket and a list of uploaded objects. These objects represent the static frontend content.

Expected result: The frontend assets are available in S3 and ready to be delivered through CloudFront.

6. CloudFront — Create Distribution

Step 1: Get started
Step 2: **Specify origin**
Step 3: Enable security
Step 4: Review and create

Specify origin

Your origin is where your content (such as a website or app) lives. CloudFront works with AWS-based origins and origins hosted on other cloud providers.

Origin type

- ☒ **Amazon S3**
Deliver static assets like files and images, statically generated websites or single page applications (SPA).
- ☐ Elastic Load Balancer
Deliver applications hosted behind ELB such as dynamic websites, web services, and APIs.
- ☐ API Gateway
Deliver API endpoints for REST APIs hosted on API Gateway.
- ☐ Elemental MediaPackage
Deliver end-to-end live events or video on demand (VOD).
- ☐ VPC origin
Deliver applications and content hosted within private VPCs, such as EC2 instances and Application Load Balancers.
- ☐ Other
Refer to any AWS or non-AWS origin through its publicly resolvable URL.

Origin

S3 origin
Choose an AWS origin, or enter your origin's domain name. [Learn more](#)

example-bucket.s3.us-east-1.amazonaws.com [Browse S3](#)

Origin path - optional
The directory path within your origin where your content is stored. [Learn more](#)

/path

Settings [info](#)
CloudFront provides default origin and cache settings based on what origin you selected. [View default settings for S3](#)

Allow private S3 bucket access to CloudFront [info](#)
CloudFront will update your S3 bucket policy to allow CloudFront to access your S3 bucket. The policy allows CloudFront to access the bucket only when the request is on behalf of the CloudFront distribution that contains the S3 origin.

☒ **Allow private S3 bucket access to CloudFront - Recommended**

Origin settings
Origin settings control how CloudFront connects to the specified origin.

What this step does: Amazon CloudFront is introduced as the content delivery layer between users and the S3 bucket.

Why it matters: CloudFront provides CDN-based delivery and becomes the public entry point for the frontend. It also provides the mechanism for using Origin Access Control with the S3 origin.

What the screenshot shows: The screenshot shows the CloudFront distribution creation/configuration interface, including the origin selection area.

Expected result: A CloudFront distribution is prepared to serve the S3-hosted frontend.

7. CloudFront Origin Configuration

CDN-for-serverless-web [Standard](#) [View metrics](#)

Details

Distribution domain name d28coesbhm601.cloudfront.net	Billing -	ARN arn:aws:cloudfront:064083568141:distribution/E3HG8RU4KUJVB	Last modified Deploying
--	--------------	---	----------------------------

General | Security | Origins | Behaviors | Error pages | Invalidations | Logging | Tags

Settings [Edit](#)

Name CDN-for-serverless-web	Alternate domain names Add domain	Standard logging Off
Description -		Cookie logging Off
Price class Use all edge locations (best performance)		Default root object -
Supported HTTP versions HTTP/2, HTTP/1.1, HTTP/1.0		Cache tag header -

Continuous deployment [info](#) [Create staging distribution](#)

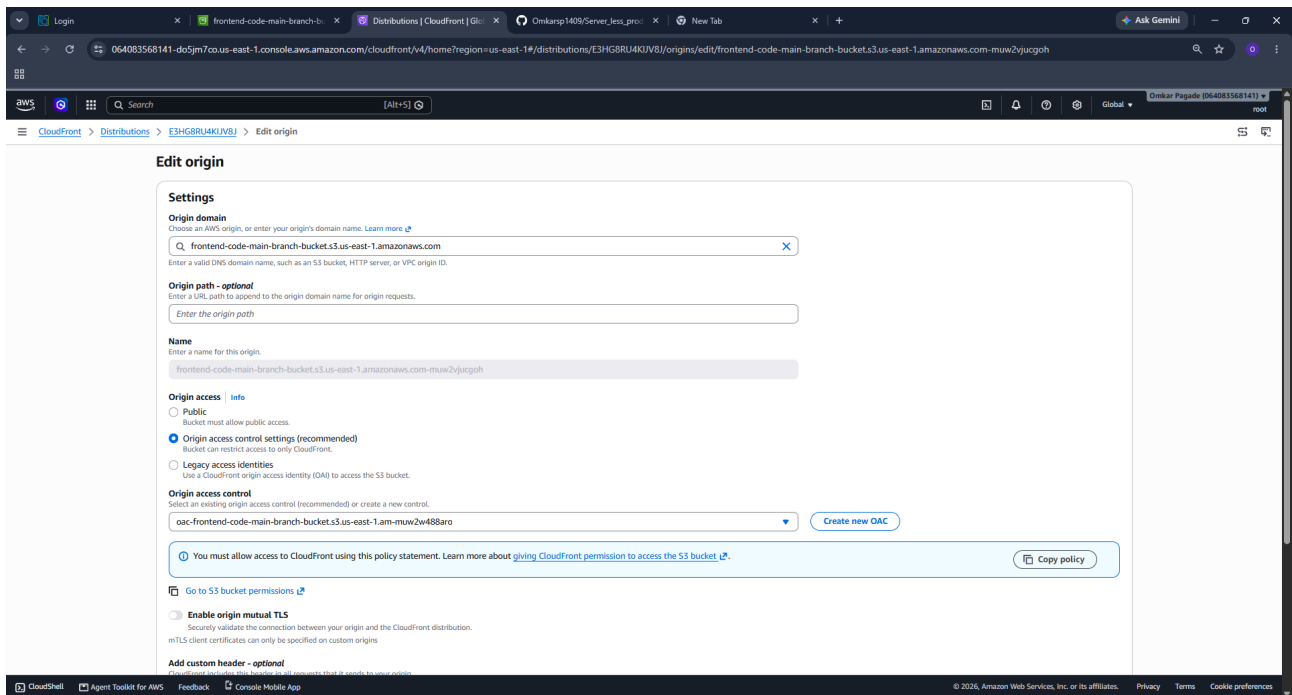
What this step does: The origin configuration identifies where CloudFront retrieves the frontend content.

Why it matters: Correct origin configuration is critical. If CloudFront points to the wrong bucket, endpoint or path, the application can return errors such as 403 AccessDenied or missing-object responses.

What the screenshot shows: The screenshot shows the CloudFront Edit origin screen with origin and access-related configuration.

Expected result: CloudFront is linked to the intended S3 origin and prepared for secure origin access.

8. Origin Access Control (OAC)



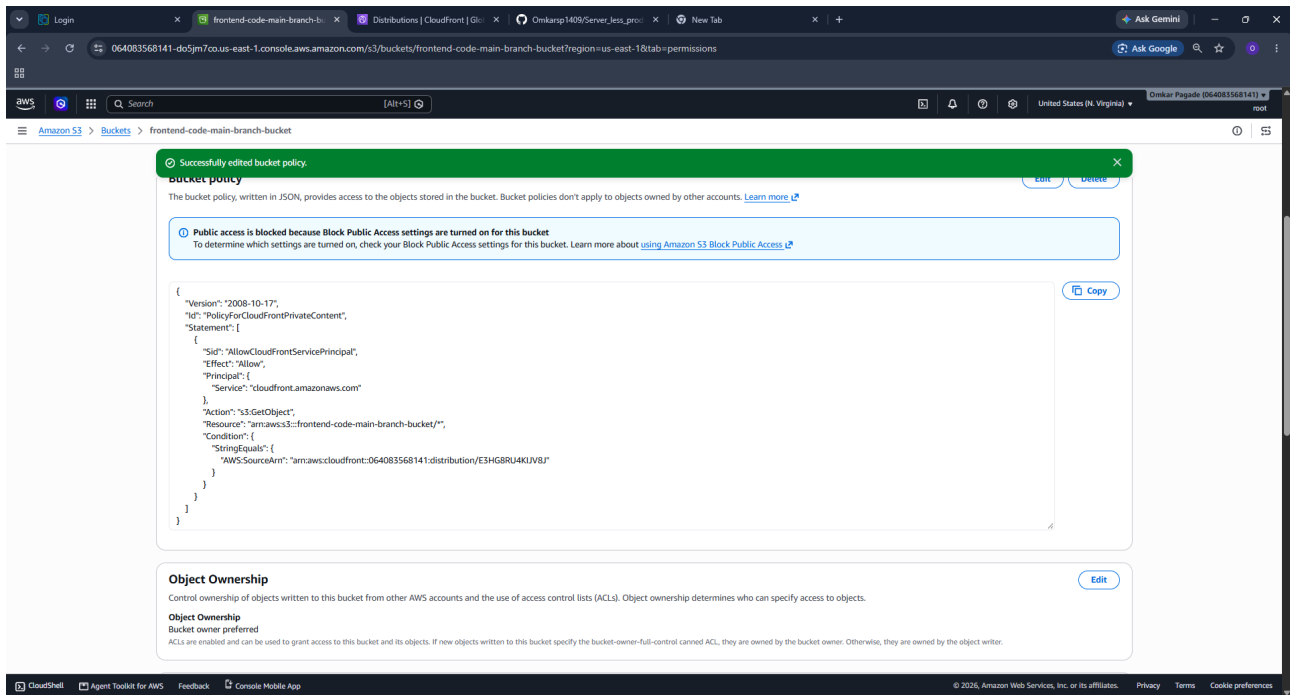
What this step does: Origin Access Control allows CloudFront to make authenticated requests to the S3 origin.

Why it matters: OAC is an important security component of this architecture. Instead of making the S3 origin broadly public, access is controlled so CloudFront can retrieve the objects.

What the screenshot shows: The screenshot shows the CloudFront origin configuration with the Origin Access Control setting selected.

Expected result: The architecture establishes CloudFront as the controlled access path to the S3 origin.

9. CloudFront Distribution — Deployment / Status



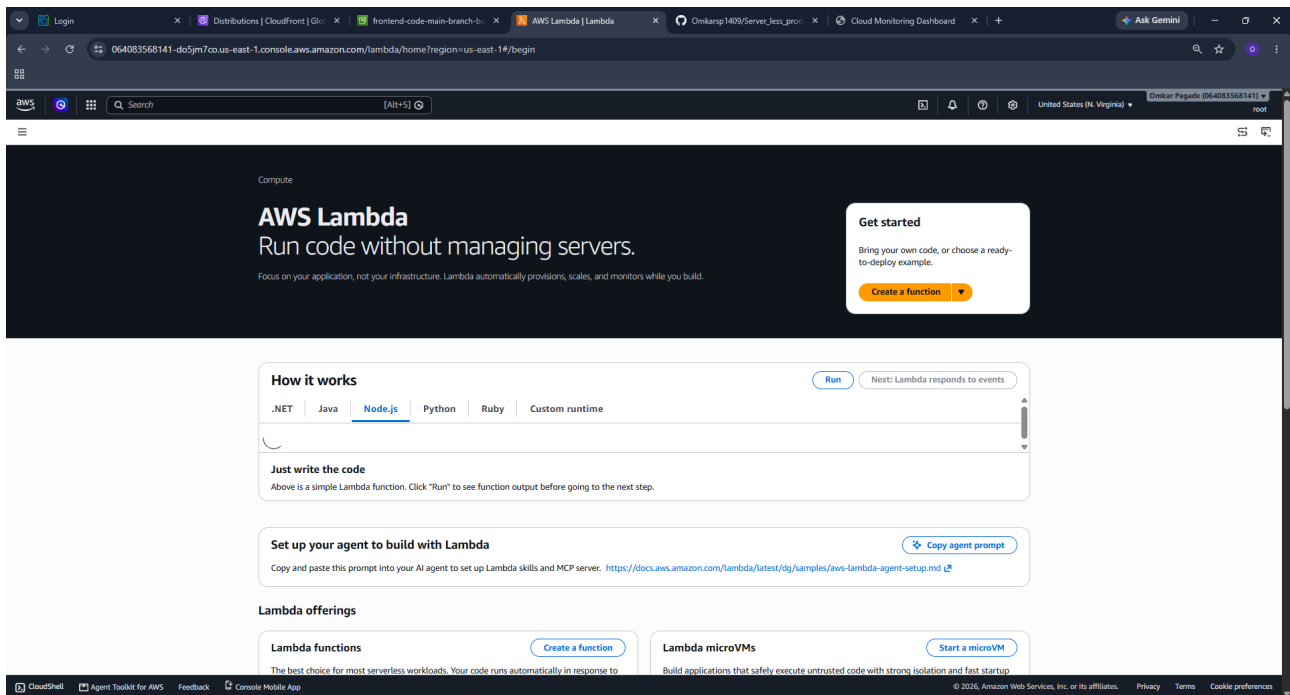
What this step does: After configuration, the CloudFront distribution needs to deploy its configuration across the CDN infrastructure.

Why it matters: A distribution can take time to become fully deployed. Testing should be performed against the deployed distribution URL after the configuration is active.

What the screenshot shows: The screenshot shows the CloudFront distribution page and its configuration/status information.

Expected result: The CDN layer is ready to serve the static frontend.

10. AWS Lambda — Serverless Backend



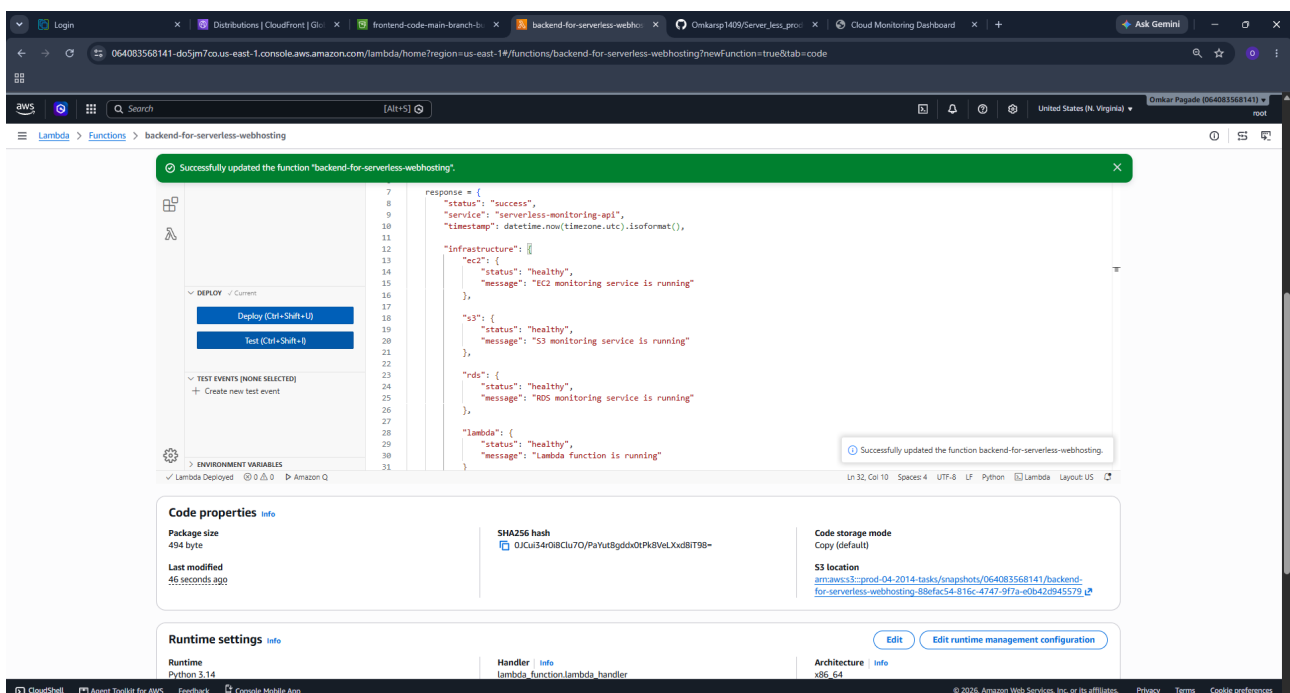
What this step does: AWS Lambda provides the backend execution layer without requiring a continuously running EC2 server.

Why it matters: Lambda is appropriate for event-driven/API-style backend logic because AWS manages the underlying compute infrastructure. The project uses a Python function named `serverless-monitoring-api`.

What the screenshot shows: The screenshot shows the AWS Lambda service page and the function creation/management interface.

Expected result: A serverless Python backend function is created for the monitoring application.

11. Lambda Function Code and Test



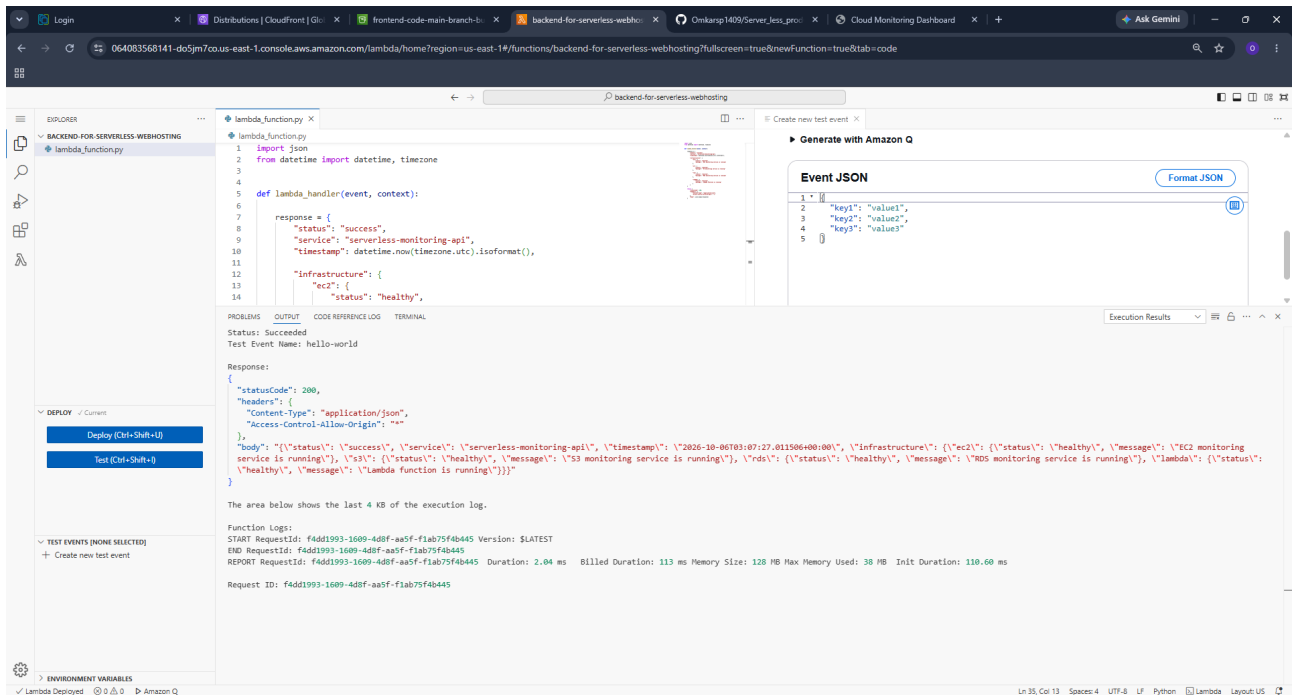
What this step does: The Lambda console is used to edit/test the function and inspect the returned result.

Why it matters: Testing confirms that the function can execute its logic and return a response. During troubleshooting, Lambda execution logs in CloudWatch are also important.

What the screenshot shows: The screenshot shows the Lambda code/test interface and the execution result area.

Expected result: The backend function can be tested independently before being consumed by the frontend.

12. Lambda + DynamoDB Data Layer



What this step does: DynamoDB provides the serverless NoSQL data layer for the monitoring workflow.

Why it matters: Using DynamoDB keeps the application serverless and avoids managing a traditional database server. Lambda can read/write the required monitoring records using IAM-controlled permissions.

What the screenshot shows: The project workflow includes DynamoDB as the backend data store. The original screenshot record is primarily focused on S3, CloudFront and Lambda; therefore the exact DynamoDB table configuration is not fully visible in the uploaded screenshots.

Expected result: The architecture has a managed persistence layer for monitoring-related application data.

13. End-to-End Request Flow

1. The user opens the CloudFront URL in a browser.
2. CloudFront receives the HTTPS request and serves the frontend content.
3. CloudFront retrieves static objects from the S3 origin through the configured Origin Access Control.
4. The JavaScript frontend invokes the backend functionality required by the monitoring application.
5. AWS Lambda executes the Python serverless backend logic.
6. Lambda uses the configured IAM permissions to access DynamoDB when the workflow requires data operations.
7. The Lambda response is returned to the application and presented to the user.
8. Operational troubleshooting can be performed using Lambda execution logs and other AWS monitoring facilities.

14. Security Notes

- Keep the S3 origin protected when using CloudFront OAC.
- Use least-privilege IAM permissions for Lambda.
- Never commit AWS access keys, secrets or passwords to GitHub.
- Restrict DynamoDB permissions to the required table and operations.
- Use CloudWatch logs to troubleshoot failed Lambda executions.

15. Troubleshooting Guide

CloudFront 403 / AccessDenied: Check the S3 bucket policy, OAC association, origin configuration, object path and CloudFront deployment status.

Frontend not loading: Verify that the requested object exists in S3 and that CloudFront is using the correct origin.

Lambda 500: Check Lambda execution logs, IAM permissions, input payload, function configuration and DynamoDB operations.

DynamoDB data-type problems: Validate the data types sent from Python and use DynamoDB-supported numeric/string representations.

16. Portfolio Summary

Built a serverless monitoring application on AWS using Amazon S3, Amazon CloudFront, CloudFront Origin Access Control, AWS Lambda and DynamoDB. The project demonstrates secure static content delivery, serverless backend execution, managed data storage and practical cloud troubleshooting.

17. Important Documentation Note

The uploaded source document is screenshot-based and does not contain machine-readable explanatory text. Therefore, the screenshot explanations in this PDF describe only what is visible in the supplied screenshots and the project workflow already established in the conversation. Where a configuration is not visible in the source screenshots, it is explicitly identified rather than presented as confirmed implementation evidence.